



## rust嵌入式开发之总结（二）Embassy的不足

原创 jxandrew 于 2024-06-24 11:13:27 发布 阅读量947 收藏 8 点赞数 6

文章标签: [rust](#) [嵌入式开发](#) [Embassy](#)

我们用 **rust** +Embassy开发的新版产品已经投产了一个多月了，经历过近距离的强干扰、连绵的阴雨天失电等考验，初步证明了整个产品体系的稳定性和可靠性。经历过开发、测试以及这段时间的运行后，我也发现了Embassy的一些问题，之前的几篇文章都在说它的好处，所以集中说一下它的问题，以帮助大全面的进行评估。

### 供应链难以维持

先概括一下我所认为的Embassy发力点：较为全面的对芯片的支持+async/await的支持。

Embassy支持了几个芯片系列，其它我没用所以无法评价，所以我只针对我使用的**STM32**来说。这里的较为全面，就STM32系列来说，应该说是全面的。

我们因为串口用的很多，所以选的是10串口的STM32F413VG，因为要用到**DMA**，所以只能用到7个串口，Embassy完全符合手册的说明，所以只CubeMX配置结果来选串口以及对应的DMA通道就可以了，没有任何的问题。

也就是说，Embassy对STM32芯片的支持是全方面的，同时还提供了自己的HAL，所有芯片的操作接口是相同的，这是非常有意义的。

但其第一个问题也就来了：**不支持GD32等对应STM32的国产芯片**。这在国产替代的当下，显然是一个非常突出的严重问题。

我们在确定rust+Embassy+STM32这条技术路线的时候，当然对供应链做过调查，确认是足以满足我们的供应的，虽然F413目前全国库存只有100多万的用量并不是很大，买上一批就够我们用上一段时间的了。

但是，架不住我们其它辅助器件的芯片断供啊！

我们现在不得不考虑全产品都要全面使用国产芯片的技术路线的巨大调整了:(

所以，Embassy的第一个问题就是如何规避供应问题，此外还要面对越来越高的国产替代要求。

### 函数 动态调用消耗巨大

在c中动态调用一个函数非常简单：预先保存好对应的函数指针，需要时直接调用就好了。

rust当然和c一样，可以使用函数指针，但这个函数指针只能调用同步函数，无法执行异步操作。

这就比较麻烦了，我们的状态机、命令行、远程控制，甚至包括debug功能，都需要异步操作，如最简单的将信息打印到控制台，就是异步的，其它定时、IO操作等等，更都必须是异步的。

我最终的解决办法就是在初始化这些模块时，提供一个spawner，各模块将这个spawner保存起来，然后在回调的同步函数指针中用这个spawner来spaw异步任务。

这样的迂回自然带来了spawner保存的开销、异步任务调用的繁琐【需要4个函数的逐步调用才能真正启动c中简单的一个函数指针的执行】，同时还带写的繁琐！

这些开销自然还可以忍受，但我在测试的时候，发现了一个巨大的bug：**这些命令行、远程控制最多只能执行三四个，再多就会崩溃了！！**

经过排查，发现这是由于Embassy启动异步任务的锅，Embassy自动生成的异步任务的调用代码是：

```
1 fn system_reboot_inner(  
2     mi: u32,  
3     purpose: DualPurpose,  
4     params: Option<BTreeMap<String, Value>>,  
5 ) -> ::embassy_executor::SpawnToken<impl Sized> {  
6     const POOL_SIZE: usize = 1;  
7     static POOL: ::embassy_executor::_export::TaskPoolRef = ::embassy_executor::_export::TaskPoolRef::new();  
8     unsafe {  
9         POOL.get::<_, POOL_SIZE>()  
10         ._spawn_async_fn(move || __system_reboot_inner_task(mi, purpose, params))  
11     }  
12 }
```

即一个任务分配一个任务池，静态的任务池，而spaw的任务是Sized的。

由于时间太紧，我没有进一步的研究Embassy的异步任务池，只能将任务池加载到ram中静态保存，任务结束也并不释放！直接导



jxandrew

关注

👍 6



★ 8

💬 3



这个bug直接导致我们对于远程控制进行了剪裁，大量的功能从远程控制中取消，同时状态机也暂停使用，虽然在第一个版本中，这是可以接受的，但致我在上篇文章中所说的我们选择rust嵌入式理由大打折扣！

### 不支持park、block\_on等std下的异步功能

Embassy实现了自己的executor，但实现的又不够完整！这主要就表现在park和block\_on上。

不支持park，任务就无法一分为二，只能一根筋的自己跑完，简单的说就是无法自己启动一个新任务来执行某些操作。但有些时候，并不适合自己执行是启动一个子任务放到后台来做【类似python、go中的 [协程](#) 】。

那有些兄弟就会说，可以启动async啊！对，可是正如我上面所说，有时我们是在一个同步函数里，是没办法执行async的。

这时就需要block\_on来帮我们在同步函数里执行async函数或闭包。

但是，Embassy的block\_on是这个样子的：

```
1 pub fn block_on<F: Future>(mut fut: F) -> F::Output {
2     // safety: we don't move the future after this line.
3     let mut fut = unsafe { Pin::new_unchecked(&mut fut) };
4
5     let raw_waker = RawWaker::new(ptr::null(), &VTABLE);
6     let waker = unsafe { Waker::from_raw(raw_waker) };
7     let mut cx = Context::from_waker(&waker);
8     loop {
9         if let Poll::Ready(res) = fut.as_mut().poll(&mut cx) {
10             return res;
11         }
12     }
13 }
```

直接一个死循环反复的调future的poll！cpu直接干满100%，只能等中断才会有反馈，看门狗的喂狗都完蛋了！

### 结语

概要之，Embassy部分实现了rust的异步机制，提供了async/await，是一个巨大的进步，最典型的就是我之前所讲解的利用await来实现应用级的临界低rust在嵌入式编程方面的门槛起到了巨大的作用。

但由于Embassy对rust异步机制支持的不完备，又严重制约了我们灵活的提供一些强大的功能:(

即，Embassy对无需动态执行任务的应用场景是非常良好的，但如果需要动态执行任务，那其显然是无法满足的。

### 使馆是一个生锈的嵌入式异步工具-Rust开发

Embassy是一个使异步/等待成为 [嵌入式开发](#) 的一流选择的项目。 Embassy Embassy是一个使async / await成为 [嵌入式开发](#) 的一流选择的项目。大使馆的板条箱定义了一！

### Rust 嵌入式开发教程之 STM32F103

这是一个关于 STM32F103 开发板使用案例，记录一些使用 [Rust](#) 语言进行学习 [嵌入式开发](#) 的示例。

3 条评论



CSDN-Ada助手 [热评](#) 恭喜你这篇博客进入【CSDN月度精选】榜单，全部的排名请看 [网页链接](#)。

...到异步[Rust](#):在嵌入式系统中的应用之旅\_embassy 操作系统

4. 异步[Rust](#):Embassy的优雅之道 Embassy框架提供了异步编程模型,使得我们可以在嵌入式系统中以极低的开销实现并发。异步编程通过等待特定事件发生(如按钮按下)对

【三万言】今年 [Rust](#) 语言出圈了!下一代系统语言 [Rust](#) 前沿报告-CSDN...

[Rust](#)是一种一旦了解就会产生使用欲望的语言,[Rust](#) 语言连续八年蝉联 stackoverflow 开发者调查问卷 最受欢迎编程语言榜首就是最好的证明。但比起“最受欢迎”的连胜纪

### rust嵌入式开发之RTICvsEmbassy

jxandrew的

RTIC和Embassy是目前rust嵌入式开发中比较热门的两个框架。本来呢，针对RTIC的移植已经完成了一小半，但在移植过程中感受到了RTIC的不足，正好跳出来全面考察

### Embassy: 革新嵌入式Rust开发的异步框架

blog.boringhex

Embassy是一个为嵌入式系统设计的异步Rust执行时和HAL(硬件抽象层)框架。提供零成本的异步抽象支持多种微控制器平台实现高效的资源利用简化嵌入式应用程序的开

### 一个rust embassy esp32c3 的练习项目

longxiangam的

基于 rust embassy esp32c3 的练习项目

### 从裸机到异步Rust：深入了解Embassy执行器

m0\_63714693的

Embassy执行器通过将异步编程模型引入嵌入式系统开发，为开发者提供了一种高效、灵活且易于维护的任务调度方式。它的无堆内存管理、高效的轮询机制、集成的定时

### W5500-EVB-Pico（与大使馆）中基于 Rust 的 TCP 套

我检查了板所连接的路由器上的 MAC 地址和分配的 IP 地址，



jxandrew

关注

👍 6



🌟 8

💬 3



Rust嵌入式开发资源精选：探索低级编程的工具和库

- Embedded Rust教学系列：包含一系列关于如何使用Rust进行嵌入式开发的教程。 - 示例项目库：如STM32 Hal库，它提供了一系列示例项目，帮助开发者理解如何在特

STM32F103开发板Rust语言嵌入式开发教程

5. Rust嵌入式生态：了解Rust嵌入式领域的生态，例如操作系统如`embassy-os`，硬件抽象层（HAL）如`cortex-m-rt`，以及Rust特有的开发模式如单线程无锁编程。6. 3

前面就学习c++和Linux系统编程就可以了吗 最新发布

- 关注Rust在嵌入式领域的应用（如`embassy`框架） - 学习ROS2与自动驾驶结合案例（Apollo系统架构） 仅掌握C++和Linux相当于学会写字但不会写文章。建议至少完

stm32f0\_cubemx\_rust  
stm32f0\_cubemx\_rust

《安富莱嵌入式周报》第261期：2022.04.11--2022.04.17

Simon223的

往期周报汇总地址：嵌入式周报 - uCOS & uCGUI & emWin & embOS & TouchGFX & ThreadX - 硬汉嵌入式论坛 - Powered by Discuz! 目录 本周更新视频教程： 视频版

祝贺！2022 Rust中文社区线上Hackathon评审结果出炉！

Rust语言学习3

完全超出预期（的优秀！）！这次黑客松共有9支队伍闯入最后的评审会。他们是：233bithttps://github.com/rustcc/hackathon2022/tree/master/233bitRust实现的合成材料

rust嵌入式开发之await

jxandrew的

当我们基于Embassy来开发嵌入式时，rust提供的async/await就是一套标准化的将并发操作进行串行化的工具

使用 Embassy 创建异步主循环

m0\_63714693的

loop {info!("tick");这里定义了一个名为run的异步任务，该任务通过宏标记。这个宏告诉 Embassy 将该函数作为任务运行。任务的核心是一个无限循环，循环内每秒钟记录

Embassy 项目常见问题解决方案

gitblog\_01195的

Embassy 项目常见问题解决方案 embassy Modern embedded framework, using Rust and async. 项目地址: https://gitcode.com/gh\_mirrors/em...

rust嵌入式开发之基于await构造应用级临界区

jxandrew的

并发编程时还有一个更重要的问题需要我们解决：资源竞争。Embassy目前还只是一个有限的运行时，还不是一个ROTS，提供不了系统级的临界区。针对这个问题，笔者

rust嵌入式开发之总结

jxandrew的

我们用rust开发的新版产品刚刚交付，已经在海上安装测试完毕并顺利投产。终于松了口气，同时也有时间和精力来做个全面的总结了

关于我们 招贤纳士 商务合作 寻求报道 400-660-0108 kefu@csdn.net 在线客服 工作时间 8:30-22:00

公安备案号11010502030143 京ICP备19004658号 京网文〔2020〕1039-165号 经营性网站备案信息 北京互联网违法和不良信息举报中心  
家长监护 网络110报警服务 中国互联网举报中心 Chrome商店下载 账号管理规范 版权与免责声明 版权申诉 出版物许可证 营业执照  
©1999-2025北京创新乐知网络技术有限公司



jxandrew

博客等级 码龄16年

236

原创

218

点赞

339

收藏

274

粉丝

关注

私信

热门文章

模糊控制-模糊是什么鬼 10419

用PythonPi实现门禁系统-示例 6323

模糊控制-模糊性知识的应用问题 4845

汉语语义理解 3290

java与python的深度融合之web界面定义

3177



jxandrew

关注

6

8

3

分

分类专栏

|                                                                                   |       |      |
|-----------------------------------------------------------------------------------|-------|------|
|  | jxTMS | 143篇 |
|  | 系统开发  | 44篇  |
|  | 业务管理  | 23篇  |
|  | 学生评价  | 1篇   |
|  | JXBiz | 41篇  |
|  | 人工智能  | 12篇  |

最新评论

- rust嵌入式开发之总结 （二）Embassy的...  
CSDN-Ada助手: 恭喜你这篇博客进入【CSDN月度精选】榜单，全部的排名请看 hi ...
- rust嵌入式开发之总结 （二）Embassy的...  
CSDN-Ada助手: 恭喜你这篇博客进入【CSDN每天值得看】榜单，全部的排名请看 ...
- rust嵌入式开发之总结 （二）Embassy的...  
jxandrew: 有兄弟指出了，gd32可以平替stm32。发现是部分型号可以，我们选的f ...
- rust嵌入式开发之总结  
CSDN-Ada助手: 恭喜你这篇博客进入【CSDN月度精选】榜单，全部的排名请看 hi ...
- rust嵌入式开发之总结  
CSDN-Ada助手: 恭喜你这篇博客进入【CSDN每天值得看】榜单，全部的排名请看 ...

大家在看

- 计算机毕业设计ssm基于java的音乐管理系统的设计与实现 基于Java与SSM框架的音乐资源管理系统开发与应用 Java环境下SSM架构的音乐信息管理平台设计与实现  
321
- 二项式分布（Binomial Distribution） 67
- 【leetcode100】单词搜索 68
- 神仙级大模型教程分享，不用感谢，请叫我活雷锋！\_大模型 学习路线！大模型学习路线（2025最新）
- UI设计中的加载动画：优化用户体验的细节

最新文章

- 现在DeepSeek为代表的AI这么厉害，程序员以后还会存在吗？
- DeepSeek驱动的数字企业
- 大模型驱动的业务自动化

|           |           |
|-----------|-----------|
| 2025年 3篇  | 2024年 10篇 |
| 2023年 68篇 | 2022年 42篇 |
| 2021年 34篇 | 2020年 36篇 |
| 2018年 3篇  | 2017年 21篇 |
| 2016年 11篇 | 2014年 2篇  |
| 2013年 6篇  |           |

供应链难以维持

函数动态调用消耗巨大

不支持park、block\_on等std下的异步功能